

Unit 08: The Bourne Again Shell and Tc Shell

CONTENTS

Objectives

Introduction

- 8.1 Startup Files
- 8.2 File Descriptors
- 8.3 Writing a Simple Shell Script
- 8.4 Job Control
- 8.5 Manipulating the Directory Stack
- 8.6 Shell Variables
- 8.7 Variable Attributes
- 8.8 Keyword Variables
- 8.9 Processes
- 8.10 History
- 8.11 Alias
- 8.12 Functions
- 8.13 Command Line Options
- 8.14 Shell Features
- 8.15 The TC Shell

Summary

Keywords

Self Assessment

Answers for Self Assessment

Review Questions

Further Readings

Objectives

After studying this unit, you will be able to:

- Understand the basics of shell, parameters and variables of a shell
- Understand the special characters
- Understand about the process
- Understand the re-execution and editing of commands
- Understand the aliases and functions in Linux
- Know how to control bash, how to enter and leave the TC shell
- Understand the features common to Bourne Again and TC Shells

Introduction

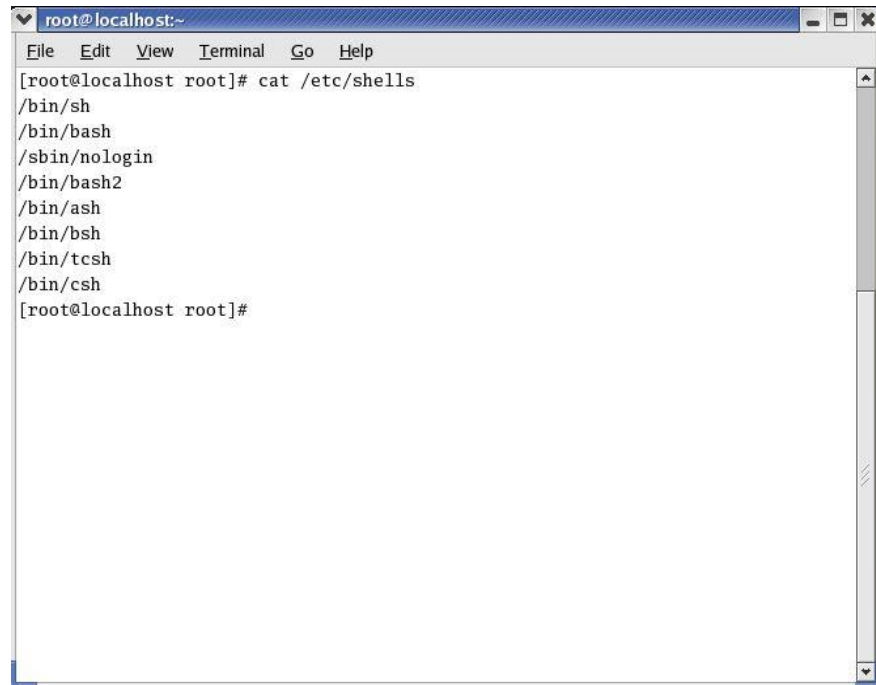
A Shell provides you with an interface to the Unix system. It gathers input from you and executes programs based on that input. When a program finish executing, it displays that program's output. Shell is an environment in which we can run our commands, programs, and shell scripts.

The Linux and Shell Scripting

There are different flavors of a shell, just as there are different flavors of operating systems. Each flavor of shell has its own set of recognized commands and functions. There are two major types of shells –

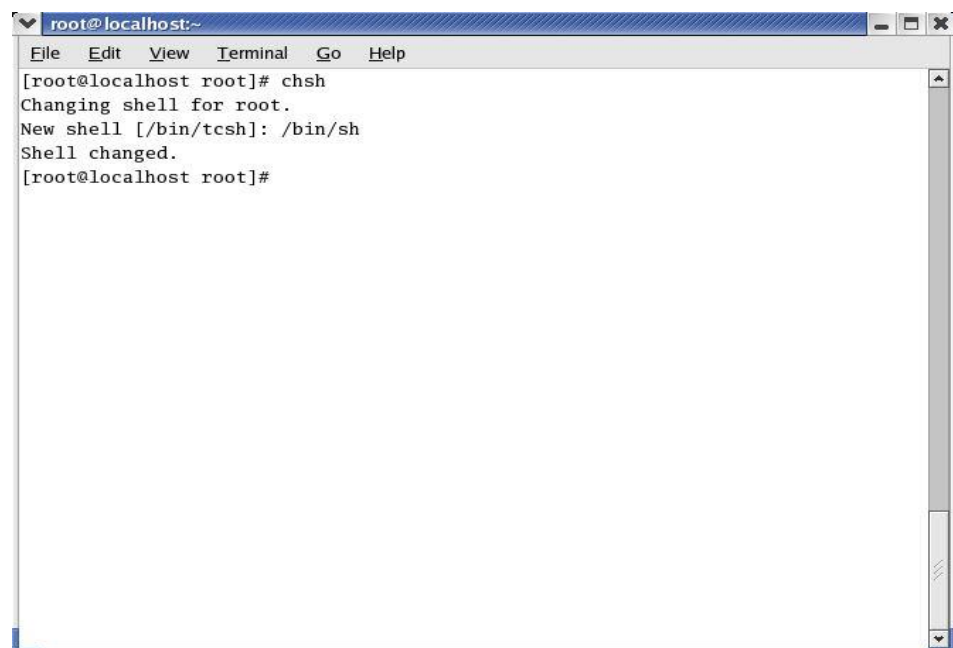
- Bourne Again shell
- TC shell

The Bourne Again Shell and TC Shell are command interpreters and high-level programming languages. As command interpreters, they process commands you enter on the command line in response to a prompt. When you use the shell as a programming language, it processes commands stored in files called shell scripts. Give the command **echo "\$SHELL"**. Under most Linux distributions, bash is the default shell. You can run any shell you like once you are logged in. When the command **cat /etc/shells** is given, it shows which shells are available to use in the system.

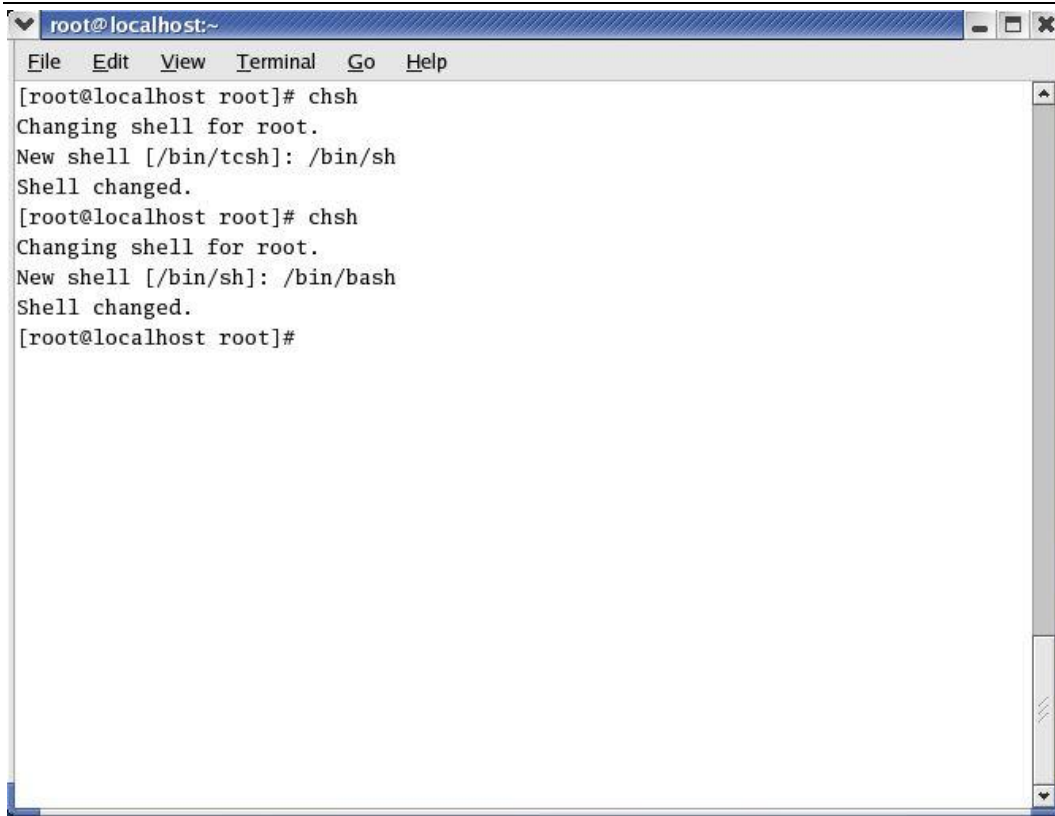


```
root@localhost:~  
File Edit View Terminal Go Help  
[root@localhost root]# cat /etc/shells  
/bin/sh  
/bin/bash  
/sbin/nologin  
/bin/bash2  
/bin/ash  
/bin/bsh  
/bin/tcsh  
/bin/csh  
[root@localhost root]#
```

Enter the name of the shell you want to use (bash, tcsh, or another shell) and press RETURN; the next prompt will be that of the new shell. Give an exit command to return to the previous shell.



```
root@localhost:~  
File Edit View Terminal Go Help  
[root@localhost root]# chsh  
Changing shell for root.  
New shell [/bin/tcsh]: /bin/sh  
Shell changed.  
[root@localhost root]#
```



```
root@localhost:~  
File Edit View Terminal Go Help  
[root@localhost root]# chsh  
Changing shell for root.  
New shell [/bin/tcsh]: /bin/sh  
Shell changed.  
[root@localhost root]# chsh  
Changing shell for root.  
New shell [/bin/sh]: /bin/bash  
Shell changed.  
[root@localhost root]#
```

8.1 Startup Files

When a shell starts, it runs startup files to initialize itself. Which files the shell runs depends on whether it is a login shell, an interactive shell that is not a login shell, or a noninteractive shell. You must have read access to a startup file to execute the commands in it.

Login Shells

The shells that you start with the `bash --login` option. Login shells are, by their nature, interactive.

- 1) **/etc/profile:** The shell first executes the commands in `/etc/profile`. A user working with root privileges can set up this file to establish system-wide default characteristics for users running bash.
- 2) **.bash_profile, .bash_login, .profile:** Next the shell looks for `~/.bash_profile`, `~/.bash_login`, and `~/.profile` (`~/` is shorthand for your home directory), in that order, executing the commands in the first of these files it finds. You can put commands in one of these files to override the defaults set in `/etc/profile`.
- 3) **.bash_logout:** When you log out, bash executes commands in the `~/.bash_logout` file. This file often holds commands that clean up after a session, such as those that remove temporary files.

Interactive Nonlogin Shells

The commands in these startup files are not executed by interactive, nonlogin shells. However, these shells inherit values from the login shell variables that are set by these startup files.

- 1) **/etc/bashrc:** Although not called by bash directly, many `~/.bashrc` files call `/etc/bashrc`. This setup allows a user working with root privileges to establish systemwide default characteristics for nonlogin bash shells.
- 2) **.bashrc:** An interactive nonlogin shell executes commands in the `~/.bashrc` file. Typically, a startup file for a login shell, such as `.bash_profile`, runs this file, so both login and nonlogin shells run the commands in `.bashrc`.

The Linux and Shell Scripting

However, the noninteractive shells inherit login shell variables that are set by these startup files. Noninteractive shells look for the environment variable BASH_ENV (or ENV if the shell is called as sh) and execute commands in the file named by this variable.

There are various commands that are used as symbols. The Bourne Again Shell uses these symbols as shown in the below table. A command can send error messages to standard error to keep them from getting mixed up with the information it sends to standard output.

Symbol	Command
()	Subshell
\$()	Command Substitution
(())	Arithmetic evaluation
\$(())	Arithmetic expression
[]	The test command
[[]]	Conditional expression; similar to [] but adds string comparisons

8.2 File Descriptors

A file descriptor is the place a program sends its output to and gets its input from. When you execute a program, Linux opens three file descriptors for the program:

- 0 (standard input), (0<)
- 1 (standard output), (1>)
- and 2 (standard error), (2>)

Opening a File Descriptor

The Bourne Again Shell opens files using the exec built in as follows:

- exec n>outfile: The line opens outfile for output and holds it open, associating it with file descriptor n.
- exec m<infile: The line opens infile for input and holds it open, associating it with file descriptor m.

Duplicating a file descriptor

The <& token duplicates an input file descriptor; >& duplicates an output file descriptor.

Redirection Operators

There are various redirection operators which can be used. These are shown in the table below.

Operator	Meaning
< filename	Redirects standard input from filename
>filename	Redirects standard output to filename unless filename exists and noclobber is set. If noclobber is not set, this redirection creates filename if it does not exist and overwrites it if it does exist.
> filename	Redirects standard output to filename, even if the file exists and noclobber is set.
>> filename	Redirects and appends standard output to filename unless filename exists and noclobber is set. If noclobber is not set, this redirection creates filename if it does not exist.
&> filename	Redirects standard output and standard error to filename.
<&m	Duplicates standard input from file descriptor m
[n] >&m	Duplicates standard output or file descriptor n if specified from file descriptor m
[n] <&-	Closes standard input or file descriptor n if specified
[n] >&-	Closes standard output or file descriptor n if specified.

8.3 Writing a Simple Shell Script

A shell script is a file that holds commands that the shell can execute. The commands in a shell script can be any commands you can enter in response to a shell prompt. For example, a command in a shell script might run a Linux utility, a compiled program, or another shell script. Like the commands you give on the command line, a command in a shell script can use ambiguous file references and can have its input or output redirected from or to a file or sent through a pipe. You can also use pipes and redirection with the input and output of the script itself.

chmod: Makes a File Executable

To execute a shell script by giving its name as a command, you must have permission to read and execute the file that contains the script. Read permission enables you to read the file that holds the script. Execute permission tells the shell and the system that the owner, group, and/or public has permission to execute the file; it implies that the content of the file is executable. When you create a shell script using an editor, the file does not typically have its execute permission set. The following example shows a file named whoson that contains a shell script:

```
$ cat whoson
```

```
date
```

```
echo "Users Currently Logged In"
```

who

\$./whoson

```
$ ls -l whoson
-rw-rw-r-- 1 max group 40 May 24 11:30 whoson

$ chmod u+x whoson
$ ls -l whoson
-rwxr--r-- 1 max group 40 May 24 11:30 whoson

$ ./whoson
Fri May 22 11:40:49 PDT 2009
Users Currently Logged In
zach pts/7 May 21 18:17
hls pts/1 May 22 09:59
sam pts/12 May 22 06:29 (bravo.example.com)
max pts/4 May 22 09:08
```

#! Specifies a Shell

You can put a special sequence of characters on the first line of a shell script to tell the operating system which shell (or other program) should execute the file. Because the operating system checks the initial characters of a program before attempting to execute it using `exec`, these characters save the system from making an unsuccessful attempt.

Begins a Comment

Comments make shell scripts and all code easier to read and maintain by you and others. The comment syntax is common to both the Bourne Again and the TC Shells.

Executing a Shell Script

A command on the command line causes the shell to fork a new process, creating a duplicate of the shell process (a subshell). The new process attempts to `exec` (execute) the command. Like `fork`, the `exec` routine is executed by the operating system (a system call).

Separating and Grouping Commands

Whether you give the shell commands interactively or write a shell script, you must separate commands from one another.

8.4 Job Control

A job is a command pipeline. For example: you run a simple job whenever you give the shell a command. If you type `date` on the command line and press RETURN, you have run a job. You can also create several jobs with multiple commands on a single command line. For example:

```
$ find . -print | sort | lpr & grep -l max /tmp/* >maxfiles&
```

```
[1] 18839
```

```
[2] 18876
```

jobs: Lists Jobs

The jobs built in lists all background jobs. Following, the sleep command runs in the background and creates a background job that jobs reports on:

```
$ sleep 60 &
```

```
[1] 7809
```

```
$ jobs
```

```
[1] + Running sleep 60 &
```

fg: Brings a Job to the Foreground

The shell assigns a job number to each command you run in the background. For each job run in the background, the shell lists the job number and PID number immediately, just before it issues a prompt:

```
$ xclock&
```

```
[1] 1246
```

```
$ date &
```

```
[2] 1247
```

```
$ Tue Dec 2 11:44:40 PST 2008
```

```
[2]+ Done date
```

```
$ find /usr -name ace -print >findout&
```

```
[2] 1269
```

```
$ jobs
```

```
[1]- Running xclock&
```

```
[2]+ Running find /usr -name ace -print >findout&
```

To move a background job to the foreground, use the fg builtin followed by the job number. Alternatively, you can give a percent sign (%) followed by the job number as a command. Either of the following commands moves job 2 to the foreground. When you move a job to the foreground, the shell displays the command it is now executing in the foreground.

```
$ fg 2
```

```
find /usr -name ace -print >findout
```

```
or
```

```
$ %2
```

```
find /usr -name ace -print >findout
```

Suspending a Job

Pressing the suspend key (usually CONTROL-Z) immediately suspends (temporarily stops) the job in the foreground and displays a message that includes the word Stopped.

```
CONTROL-Z
```

```
[2]+ Stopped find /usr -name ace -print >findout
```

bg: Sends a Job to the Background

To move the foreground job to the background, you must first suspend the job using CONTROL-Z. You can then use the bg built in to resume execution of the job in the background.

```
$ bg
```

```
[2]+ find /usr -name ace -print >findout&
```

8.5 Manipulating the Directory Stack

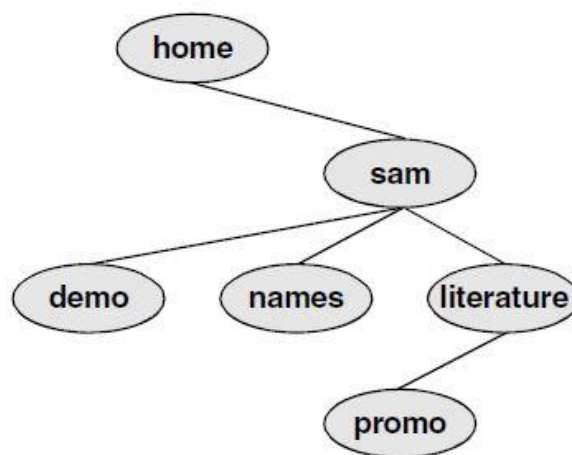
The Bourne Again Shell allows you to store a list of directories you are working with, enabling you to move easily among them. This list is referred to as a stack. It implements LIFO rule.

dirs: Displays the Stack

The `dirs` builtin displays the contents of the directory stack. If you call `dirs` when the directory stack is empty, it displays the name of the working directory: `$ dirs`

`~/literature`

The `dirs` builtin uses a tilde (`~`) to represent the name of the home directory.



pushd: Pushes a Directory on the Stack

When you supply the `pushd` (push directory) builtin with one argument, it pushes the directory specified by the argument on the stack, changes directories to the specified directory, and displays the stack.

```
$ pushd ../demo
```

```
~/demo ~/literature
```

```
$ pwd
```

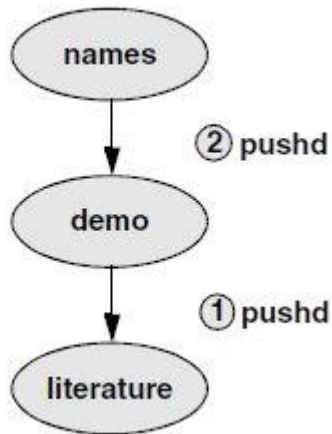
```
/home/sam/demo
```

```
$ pushd ../names
```

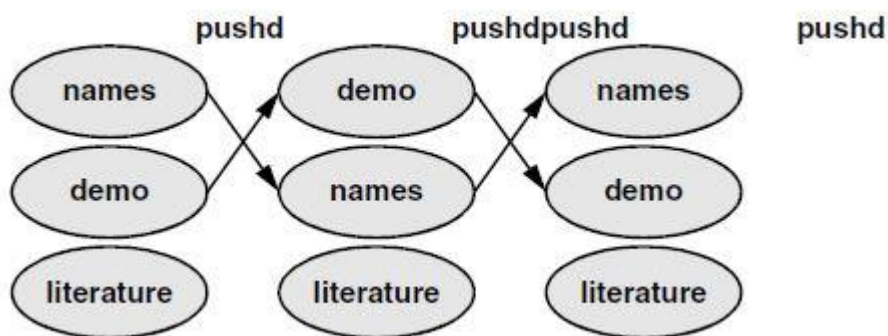
```
~/names ~/demo ~/literature
```

```
$ pwd
```

```
/home/sam/names
```

When you use `pushd` without an argument, it swaps the top two directories on the stack, makes the new top directory (which was the second directory) the new working directory, and displays the stack



Using `pushd` in this way, you can easily move back and forth between two directories. You can also use `cd -` to change to the previous directory, whether you have explicitly created a directory stack. To access another directory in the stack, call `pushd` with a numeric argument preceded by a plus sign. The directories in the stack are numbered starting with the top directory, which is number 0.

popd: Pops a Directory Off the Stack

To remove a directory from the stack, use the `popd` (pop directory) builtin.

\$ dirs

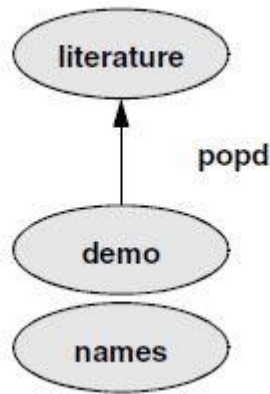
~/literature ~/demo ~/names

\$ popd

~/demo ~/names

\$ pwd

/home/sam/demo



To remove a directory other than the top one from the stack, use `popd` with a numeric argument preceded by a plus sign.

```
$ dirs
```

```
~/literature ~/demo ~/names
```

```
$ popd +1
```

```
~/literature ~/names
```

8.6 Shell Variables

Within a shell, a shell parameter is associated with a value that is accessible to the user. There are several kinds of shell parameters. The parameters whose names consist of letters, digits, and underscores are often referred to as shell variables, or simply variables. A variable name must start with a letter or underscore, not with a number. Thus `A76`, `MY_CAT`, and `__X__` are valid variable names, whereas `69TH_STREET` (starts with a digit) and `MY-NAME` (contains a hyphen) are not.

Keyword variables

Keyword shell variables (or simply keyword variables) have special meaning to the shell and usually have short, mnemonic names. When you start a shell (by logging in, for example), the shell inherits several keyword variables from the environment. Among these variables are `HOME`, which identifies your home directory, and `PATH`, which determines which directories the shell searches and in what order to locate commands that you give the shell. The shell creates and initializes (with default values) other keyword variables when you start it. Still other variables do not exist until you set them. You can change the values of most keyword shell variables. It is usually not necessary to change the values of keyword variables initialized in the `/etc/profile` or `/etc/csh.cshrc` systemwide startup files. If you need to change the value of a bash keyword variable, do so in one of your startup files. Just as you can make user-created variables global, so you can make keyword variables global—a task usually done automatically in startup files. You can also make a keyword variable read only.

Positional and special parameters

The names of positional and special parameters do not resemble variable names. Most of these parameters have one-character names (for example, `1`, `?`, and `#`) and are referenced (as are all variables) by preceding the name with a dollar sign (`$1`, `$?`, and `$#`). The values of these parameters reflect different aspects of your ongoing interaction with the shell. Whenever you give a command, each argument on the command line becomes the value of a positional parameter. Positional parameters enable you to access command-line arguments, a capability that you will often require when you write shell scripts. The `set` builtin enables you to assign values to positional parameters. Other frequently needed shell script values, such as the name of the last command executed, the number of command-line arguments, and the status of the most recently executed command, are available as special parameters. You cannot assign values to special parameters.

```
$ person=max
$ echo person
person
$ echo $person
max
```

User-created variables

Shell variables that you name and assign values to are user-created variables. You can change the values of user-created variables at any time, or you can make them read only so that their values cannot be changed.

User created global variables

You can also make user-created variables global. A global variable (also called an environment variable) is available to all shells and other programs you fork from the original shell. One naming convention is to use only uppercase letters for global variables and to use mixed-case or lowercase letters for other variables.

Assigning values

To assign a value to a variable in the Bourne Again Shell, use the following syntax: **VARIABLE=value**. There can be no whitespace on either side of the equal sign (=). An example assignment follows: **\$ myvar=abc**. Under the TC Shell the assignment must be preceded by the word **set** and the SPACES on either side of the equal sign are optional: **\$ set myvar = abc**. The Bourne Again Shell permits you to put variable assignments on a command line. This type of assignment creates a variable that is local to the command shell—that is, the variable is accessible only from the program the command runs.

```
$ cat my_script
echo $TEMPDIR
$ TEMPDIR=/home/sam/temp ./my_script
/home/sam/temp
$ echo $TEMPDIR
$
```

The `my_script` shell script displays the value of `TEMPDIR`. The following command runs `my_script` with `TEMPDIR` set to `/home/sam/temp`. The `echo` builtin shows that the interactive shell has no value for `TEMPDIR` after running `my_script`. If `TEMPDIR` had been set in the interactive shell, running `my_script` in this manner would have had no effect on its value.

Parameter substitution

Because the `echo` builtin copies its arguments to standard output, you can use it to display the values of variables.

```
$ person=max
$ echo person
person
$ echo $person
max
```

The Linux and Shell Scripting

The second line of the example shows that `person` does not represent `max`. Instead, the string `person` is echoed as `person`. The shell substitutes the value of a variable only when you precede the name of the variable with a dollar sign (`$`). Because of the leading `$`, the shell recognizes that `$person` is the name of a variable, substitutes the value of the variable, and passes that value to `echo`.

Quoting the `$`

You can prevent the shell from substituting the value of a variable by quoting the leading `$`. Double quotation marks do not prevent the substitution; single quotation marks or a backslash (`\`) do.

```
$ echo $person
```

```
max
```

```
$ echo "$person"
```

```
max
```

```
$ echo 'person'
```

```
person
```

```
$ echo \person
```

```
person
```

Spaces

To assign a value that contains SPACES or TABS to a variable, use double quotation marks around the value.

```
$ person="max and zach"
```

```
$ echo $person
```

```
max and zach
```

```
$ person=max and zach
```

```
bash: and: command not found
```

If you do not quote the variable, the shell collapses each string of blank characters into a single SPACE before passing the variable to the utility:

```
$ person="max and zach"
```

```
$ echo $person
```

```
max and zach
```

```
$ echo "$person"
```

```
max and zach
```

Pathname expansion in assignments

When you execute a command with a variable as an argument, the shell replaces the name of the variable with the value of the variable and passes that value to the program being executed. If the value of the variable contains a special character, such as `*` or `?`, the shell may expand that variable.

```
$ memo=max*
```

```
$ echo "$memo"
```

```
max*
```

The first line assigns the string `max*` to the variable `memo`. The Bourne Again Shell does not expand the string because `bash` does not perform pathname expansion when it assigns a value to a variable. All shells process a command line in a specific order. Within this order `bash` (but not `tcsh`)

Unit 08: The Bourne Again Shell and TC Shell

expands variables before it interprets commands. The echo command line, the double quotation marks quote the asterisk (*) in the expanded value of \$memo and prevent bash from performing pathname expansion on the expanded memo variable before passing its value to the echo command.

```
$ ls
```

```
max.report
```

```
max.summary
```

```
$ echo $memo
```

```
max.reportmax.summary
```

All shells interpret special characters as special when you reference a variable that contains an unquoted special character. The shell expands the value of the memo variable because it is not quoted. Here the shell expands the **\$memo variable to max***, expands **max*** to **max.report** and **max.summary**, and passes these two values to echo.

User-Created Variables - unset: Removes a Variable

Unless you remove a variable, it exists as long as the shell in which it was created exists. To remove the value of a variable but not the variable itself, assign a null value to the variable (use set person = in tcsh):

```
$ person=
```

```
$ echo $person
```

```
$
```

You can remove a variable using the unset builtin. The following command removes the variable person:

```
$ unset person
```

8.7 Variable Attributes

readonly: Makes the Value of a Variable Permanent

You can use the readonly builtin (not in tcsh) to ensure that the value of a variable cannot be changed.

```
$ person=zach
```

```
$ echo $person
```

```
zach
```

```
$ readonly person
```

```
$ person=helen
```

```
bash: person: readonly variable
```

If you use the readonly builtin without an argument, it displays a list of all readonly shell variables. This list includes keyword variables that are automatically set as readonly as well as keyword or user-created variables that you have declared as readonly.

declare and typeset: Assign Attributes to Variables

The declare and typeset builtins (two names for the same command, neither of which is available in tcsh) set attributes and values for shell variables.

Attribute	Meaning
-a	Declares a variable as an array
-f	Declares a variable to be a function name
-I	Declares a variable to be of type integer
-r	Makes a variable readonly
-x	Exports a variable, makes it global

\$ **declare person1=max**: The first line declares person1 and assigns it a value of max. This command has the same effect with or without the word declare.

\$ **declare -r person2=zach**: The readonly and export builtins are synonyms for the commands declare -r and declare -x, respectively.

\$ **declare -rx person3=helen**

\$ **declare -x person4**: You can declare a variable without assigning a value to it, as the preceding declaration of the variable person4 illustrates. This declaration makes person4 available to all subshells (i.e., makes it global). Until an assignment is made to the variable, it has a null value. You can list the options to declare separately in any order. The following is equivalent to the preceding declaration of person3: \$ declare -x -r person3=helen. Use the + character in place of - when you want to remove an attribute from a variable. You cannot remove the readonly attribute. After the following command is given, the variable person3 is no longer exported but it is still readonly. \$ declare +x person3. You can use typeset instead of declare.

Listing variable attributes

Without any arguments or options, declare lists all shell variables. The same list is output when you run set without any arguments.

```
$ declare -r
declare -ar BASH_VERSINFO='([0]="3" [1]="2" [2]="39" [3]="1" ... )'
declare -ir EUID="500"
declare -ir PPID="936"
declare -r SHELLOPTS="braceexpand:emacs:hashall:histexpand:history:..."
declare -ir UID="500"
declare -r person2="zach"
declare -rx person3="helen"
```

If you use a declare builtin with options but no variable names as arguments, the command lists all shell variables that have the indicated attributes set. For example, the command declare -r displays a list of all read only shell variables. This list is the same as that produced by the read only command without any arguments.

Integer

By default the values of variables are stored as strings. When you perform arithmetic on a string variable, the shell converts the variable into a number, manipulates it, and then converts it back to a string. A variable with the integer attribute is stored as an integer. Assign the integer attribute as follows:

```
$ declare -i COUNT
```

8.8 Keyword Variables

Keyword variables either are inherited or are declared and initialized by the shell when it starts. You can assign values to these variables from the command line or from a startup file. Typically, you want these variables to apply to all subshells you start as well as to your login shell. For those variables not automatically exported by the shell, you must use `export` or `setenv` to make them available to child shells.

HOME: Your Home Directory

By default, your home directory is the working directory when you log in. Your home directory is established when your account is set up; under Linux its name is stored in the `/etc/passwd` file.

```
$ pwd
/home/max/laptop
$ echo $HOME
/home/max
$ cd
$ pwd
/home/max
```

When you log in, the shell inherits the pathname of your home directory and assigns it to the variable `HOME`. When you give a `cd` command without an argument, `cd` makes the directory whose name is stored in `HOME` the working directory. This example shows the value of the `HOME` variable and the effect of the `cd` builtin. After you execute `cd` without an argument, the pathname of the working directory is the same as the value of `HOME`: your home directory.

Tilde

The shell uses the value of `HOME` to expand pathnames that use the shorthand tilde (`~`) notation to denote a user's home directory. It uses `ls` to list the files in Max's laptop directory, which is a subdirectory of his home directory:

```
$ echo ~
/home/max
$ ls ~/laptop
tester count lineup
```

PATH: Where the Shell Looks for Programs

If the file with the pathname you specified does not exist, the shell reports command not found. If the file exists as specified but you do not have execute permission for it, or in the case of a shell script you do not have read and execute permission for it, the shell reports Permission denied. If you give a simple filename as a command, the shell searches through certain directories (your search path) for the program you want to execute. It looks in several directories for a file that has the same name as the command and that you have execute permission for (a compiled program) or read and execute permission for (a shell script). The `PATH` shell variable controls this search. The

The Linux and Shell Scripting

system directories include `/bin` and `/usr/bin` and other directories appropriate to the local system. The `PATH` variable specifies the directories in the order the shell should search them. Each directory must be separated from the next by a colon. The following command sets `PATH` so that a search for an executable file starts with the `/usr/local/bin` directory. If it does not find the file in this directory, the shell looks next in `/bin`, and then in `/usr/bin`. If the search fails in those directories, the shell looks in the `~/bin` directory, a subdirectory of the user's home directory. Finally, the shell looks in the working directory. The exporting `PATH` makes its value accessible to subshells: `$ export PATH=/usr/local/bin:/bin:/usr/bin:~/bin`: A null value in the string indicates the working directory.

MAIL: Where Your Mail Is Kept

The `MAIL` variable (mail under `tcsh`) contains the pathname of the file that holds your mail (your mailbox, usually `/var/mail/name`, where `name` is your username). If `MAIL` is set and `MAILPATH` (next) is not set, the shell informs you when mail arrives in the file specified by `MAIL`. In a graphical environment you can unset `MAIL` so the shell does not display mail reminders in a terminal emulator window. The `MAIL` variable and other mail-related shell variables do not do anything unless you have a local mail server.

- **MAILPATH:** The `MAILPATH` variable (not available under `tcsh`) contains a list of filenames separated by colons. If this variable is set, the shell informs you when any one of the files is modified.
- **MAILCHECK:** This variable (not available under `tcsh`) specifies how often, in seconds, the shell checks for new mail. The default is 60 seconds. If you set this variable to zero, the shell checks before each prompt.

PS1: User Prompt (Primary)

The default Bourne Again Shell prompt is a dollar sign (`$`). When you run `bash` with root privileges, `bash` typically displays a pound sign (`#`) prompt. The `PS1` variable holds the prompt string that the shell uses to let you know that it is waiting for a command. When you change the value of `PS1` or prompt, you change the appearance of your prompt. You can customize the prompt displayed by `PS1`. For example, the assignment

```
$ PS1="[u@\h \W \!]"
```

displays the following prompt:

```
[user@host directory event]$
```

where `user` is the username, `host` is the hostname up to the first period, `directory` is the basename of the working directory, and `event` is the event number of the current command.

Symbol	Display in Prompt
<code>\\$</code>	<code>#</code> is the user is running with root privileges; otherwise, <code>\$</code>
<code>\w</code>	Pathname of the working directory
<code>\W</code>	Basename of the working directory
<code>\!</code>	Current event (history) number

Unit 08: The Bourne Again Shell and TC Shell

\d	Date in Weekday Month Date format
\h	Machine hostname, without the domain
\H	Full machine hostname, including the domain
\u	Username of the current user
\@	Current time of day in 12-hour, AM/PM format
\T	Current time of day in 12-hour HH:MM:SS format
\A	Current time of day in 24-hour HH:MM format
\t	Current time of day in 24-hour HH:MM:SS format

PS2: User Prompt (Secondary)

The PS2 variable holds the secondary prompt. On the first line, an unclosed quoted string follows echo. The shell assumes the command is not finished and, on the second line, gives the default secondary prompt (>). This prompt indicates the shell is waiting for the user to continue the command line. The shell waits until it receives the quotation mark that closes the string. Only then does it execute the command:

```
$ echo "demonstration of prompt string
```

```
>2"
```

```
demonstration of prompt string
```

```
2
```

```
$ PS2="secondary prompt: "
```

```
$ echo "this demonstrates
```

```
secondary prompt: prompt string 2"
```

```
this demonstrates
```

```
prompt string 2
```

The second command changes the secondary prompt to secondary prompt: followed by a SPACE. A multiline echo demonstrates the new prompt.

PS3: Menu Prompt

The PS3 variable holds the menu prompt for the select control structure.

PS4: Debugging Prompt

The PS4 variable holds the bash debugging symbol.

Keyword Variables

Variable	Value
BASH_ENV	The pathname of the startup file for noninteractive shells
CDPATH	The cd search path
COLUMNS	The width of the display used by select
FCEDIT	The name of the editor that fc uses by default
HISTFILE	The pathname of the file that holds the history list
HISTFILESIZE	The maximum number of entries saved in HISTFILE
HISTSIZE	The maximum number of entries saved in the history list
HOME	The pathname of the user's home directory; used as the default argument for cd and in tilde expansion
IFS	Internal Field Separator; used for word splitting
INPUTRC	The pathname of the Readline startup file
LANG	The locale category when that category is not specifically set with an LC_* variable
LC_*	A group of variables that specify locale categories including LC_COLLATE , LC_CTYPE , LC_MESSAGES , and LC_NUMERIC ; use the locale builtin to display a complete list with values
LINES	The height of the display used by select
MAIL	The pathname of the file that holds a user's mail
MAILCHECK	How often, in seconds, bash checks for mail
MAILPATH	A colon-separated list of file pathnames that bash checks for mail in
PATH	A colon-separated list of directory pathnames that bash looks for commands

Unit 08: The Bourne Again Shell and TC Shell

PROMPT_COMMAND	A command that bash executes just before it displays the primary prompt
PS1	Prompt String 1; the primary prompt
PS2	Prompt String 2; the secondary prompt
PS3	The prompt issued by select
PS4	The bash debugging symbol
REPLY	Holds the line that read accepts also used by select

Special Characters

Characters	Use
NEWLINE	Initiates the execution of a command page
;	Separates commands
()	Group commands for execution by a subshell or identifies a function
(())	Expands an arithmetic expression
&	Executes a command in the background
	Pipe
>	Redirects standard output
>>	Appends standard output
<	Redirects standard input
<<	Here document
*	Any string of zero or more characters in an ambiguous file reference
?	Any single character in an ambiguous file reference
\	Quotes the following character

'	Quotes a string, preventing all substitution
"	Quotes a string, allowing only variable and command substitution
'...'	Performs command substitution
[]	Character class in an ambiguous file reference
\$	Reference a variable
.	Executes a command
#	Begins a comment
{ }	Surrounds the contents of a function
:	Returns true
&&	Boolean AND
	Boolean OR
!	Boolean NOT
\$()	Performs command substitution
[]	Evaluates an arithmetic expression

8.9 Processes

A process is the execution of a command by the Linux kernel. The shell that starts when you log in is a command, or a process, like any other. When you give the name of a Linux utility on the command line, you initiate a process. When you run a shell script, another shell process is started and additional processes are created for each command in the script. Depending on how you invoke the shell script, the script is run either by the current shell or, more typically, by a subshell (child) of the current shell. A process is not started when you run a shell builtin, such as `cd`.

Process Structure

Like the file structure, the process structure is hierarchical, with parents, children, and even a root. A parent process forks a child process, which in turn can fork other processes. The term `fork` indicates that, as with a fork in the road, one process turns into two. Initially the two forks are identical except that one is identified as the parent and one as the child. You can also use the term `spawn`; the words are interchangeable. The operating system routine, or system call, that creates a new process is named `fork()`. When Linux begins execution when a system is started, it starts `init`, a single process called a spontaneous process, with PID number 1. This process holds the same position in the process structure as the root directory does in the file structure: It is the ancestor of all processes the system and users work with. When a command-line system is in multiuser mode,

Unit 08: The Bourne Again Shell and TC Shell

init runs getty or mingetty processes, which display login: prompts on terminals. When a user responds to the prompt and presses RETURN, getty hands control over to a utility named login, which checks the username and password combination. After the user logs in, the login process becomes the user's shell process.

Process Identification

Linux assigns a unique PID number at the inception of each process. As long as a process exists, it keeps the same PID number. During one session the same process is always executing the login shell. When you fork a new process—for example, when you use an editor—the PID number of the new (child) process is different from that of its parent process. When you return to the login shell, it is still being executed by the same process and has the same PID number as when you logged in.

- **\$ sleep 10 &**

[1] 22789

- **\$ ps -f**

UID PID PPID C STIME TTY TIME CMD

max 21341 21340 0 10:42 pts/16 00:00:00 bash

max 22789 21341 0 17:30 pts/16 00:00:00 sleep 10

max 22790 21341 0 17:30 pts/16 00:00:00 ps -f

The following example shows that the process running the shell forked (is the parent of) the process running ps. When you call it with the `-f` option, ps displays a full listing of information about each process. The line of the ps display with bash in the CMD column refers to the process running the shell. The column headed by PID identifies the PID number. The column headed PPID identifies the PID number of the parent of the process. From the PID and PPID columns you can see that the process running the shell (PID 21341) is the parent of the process running sleep (PID 22789). The parent PID number of sleep is the same as the PID number of the shell (21341). A second pair of sleep and ps `-f` commands shows that the shell is still being run by the same process but that it forked another process to run sleep:

- **\$ sleep 10 &**

[1] 22791

- **\$ ps -f**

UID PID PPID C STIME TTY TIME CMD

max 21341 21340 0 10:42 pts/16 00:00:00 bash

max 22791 21341 0 17:31 pts/16 00:00:00 sleep 10

max 22792 21341 0 17:31 pts/16 00:00:00 ps -f

You can also use pstree (or ps `--forest`, with or without the `-e` option) to see the parent-child relationship of processes

```
$ pstree -p
init(1)--acpid(1395)
  |-atd(1758)
  |-crond(1702)
  ...
  |-kdeinit(2223)--firefox(8914)---run-mozilla.sh(8920)---firefox-bin(8925)
  |
  |   |-gaim(2306)
  |   |-qqview(14062)
  |   |-kdeinit(2228)
  |   |-kdeinit(2294)
  |   |-kdeinit(2314)--bash(2329)---ssh(2561)
  |   |   |-bash(2339)
  |   |   '---bash(15821)---bash(16778)
  |   |-kdeinit(16448)
  |   |-kdeinit(20888)
  |   |-oclock(2317)
  |   '---pam-panel-icon(2305)---pam_timestamp_c(2307)
  ...
  |-login(1823)---bash(20986)--pstree(21028)
  |                                     '---sleep(21026)
  ...
```

The line that starts with `-kdeinit` shows a graphical user running many processes, including `firefox`, `gaim`, and `oclock`. The line that starts with `-login` shows a textual user running `sleep` in the background and running `pstree` in the foreground.

Executing a Command

When you give the shell a command, it usually forks [spawns using the `fork()` system call] a child process to execute the command. While the child process is executing the command, the parent process sleeps [implemented as the `sleep()` system call]. While a process is sleeping, it does not use any computer time; it remains inactive, waiting to wake up. When the child process finishes executing the command, it tells its parent of its success or failure via its exit status and then dies. The parent process (which is running the shell) wakes up and prompts for another command. When you run a process in the background by ending a command with an ampersand (&), the shell forks a child process without going to sleep and without waiting for the child process to run to completion. The parent process, which is executing the shell, reports the job number and PID number of the child process and prompts for another command. The child process runs in the background, independent of its parent. Although the shell forks a process to run most of the commands you give it, some commands are built into the shell. The shell does not need to fork a process to run builtins.

Variables Within a given process, such as your login shell or a subshell, you can declare, initialize, read, and change variables. By default, however, a variable is local to a process. When a process forks a child process, the parent does not pass the value of a variable to the child. You can make the value of a variable available to child processes (global) by using the `export` builtin under `bash` or the `setenv` builtin under `tcsh`.

8.10 History

The history mechanism, a feature adapted from the C Shell, maintains a list of recently issued command lines, also called events. There are various features of this:

- It provides a quick way to reexecute any of the events in the list.
- Execute variations of previous commands and reuses arguments from them.
- Replicates complicated commands and arguments that you used earlier in this login session or in a previous one.
- Enter a series of commands that differ from one another in minor ways.
- Serves as a record of what you have done.
- Keeps a record of a procedure that involved a series of commands.

Variables That Control History

The TC Shell's history mechanism is similar to bash's but uses different variables and has other differences.

Variable	Default	Function
HISTSIZE	500 events	Maximum number of events saved during a session
HISTFILE	~/ .bash_history	Location of the history file
HISTFILESIZE	500 events	Maximum number of events saved between session

- **HISTSIZE:** The value of the HISTSIZE variable determines the number of events preserved in the history list during a session. A value in the range of 100 to 1,000 is normal.
- **HISTFILE:** When you exit from the shell, the most recently executed commands are saved in the file whose name is stored in the HISTFILE variable (the default is ~/ .bash_history). The next time you start the shell, this file initializes the history list.
- **HISTFILESIZE:** The value of the HISTFILESIZE variable determines the number of lines of history saved in HISTFILE.

The Bourne Again Shell assigns a sequential event number to each command line. You can display this event number as part of the bash prompt by including \! in PS1. Give the following command manually, or place it in `~/ .bash_profile` to affect future sessions, to establish a history list of the 100 most recent events: `$ HISTSIZE=100`. The following command causes bash to save the 100 most recent events across login sessions: `$ HISTFILESIZE=100`. After you set HISTFILESIZE, you can log out and log in again, and the 100 most recent events from the previous login session will appear in your history list. Give the command **history** to display the events in the history list. This list is ordered so that the oldest events appear at the top. A tcsh history list includes the time the command was executed. The following history list includes a command to modify the bash prompt so it displays the history event number. The last event in the history list is the history command that displayed the list.

```
32 $ history | tail
23 PS1="\! bash$ "
24 ls -l
25 cat temp
26 rm temp
27 vim memo
28 lpr memo
29 vim memo
30 lpr memo
31 rm memo
32 history | tail
```

As you run commands and your history list becomes longer, it may run off the top of the screen when you use the history builtin. Pipe the output of history through less to browse through it, or give the command `history 10` or `history | tail` to look at the ten most recent commands.

History alias

Creating the following aliases makes working with history easier. The first allows you to give the command `h` to display the ten most recent events. The second alias causes the command `hg` string to display all events in the history list that contain string. Put these aliases in your `~/.bashrc` file to make them available each time you log in.

- `$ alias 'h=history | tail'`
- `$ alias 'hg=history | grep'`

Reexecuting and Editing Commands

You can reexecute any event in the history list. This feature can save you time, effort, and aggravation. Not having to reenter long command lines allows you to reexecute events more easily, quickly, and accurately than you could if you had to retype the command line in its entirety. You can recall, modify, and reexecute previously executed events in three ways:

- `fc` builtin,
- exclamation point commands ,
- Readline Library

fc: Displays, Edits, and Reexecutes Commands

The `fc` (fix command) builtin (not in `tcsh`) enables you to display the history list and to edit and reexecute previous commands. It provides many of the same capabilities as the command-line editors. When you call `fc` with the `-l` option, it displays commands from the history list. Without any arguments, `fc -l` lists the 16 most recent commands in a numbered list, with the oldest appearing first:

```
$ fc -l
1024 cd
1025 view calendar
1026 vim letter.adams01
1027 aspell -c letter.adams01
1028 vim letter.adams01
1029 lpr letter.adams01
1030 cd ../memos
1031 ls
1032 rm *0405
1033 fc -l
1034 cd
1035 whereisaspell
1036 man aspell
1037 cd /usr/share/doc/*aspell*
1038 pwd
1039 ls
1040man-htm
```

The `fc` builtin can take zero, one, or two arguments with the `-l` option. The arguments specify the part of the history list to be displayed: `fc -l [first [last]]`. The `fc` builtin lists commands beginning with the most recent event that matches first. The argument can be an event number, the first few characters of the command line, or a negative number, which is taken to be the *n*th previous command. Without `last`, `fc` displays events through the most recent. If you include `last`, `fc` displays

Unit 08: The Bourne Again Shell and TC Shell

commands from the most recent event that matches first through the most recent event that matches last. The next command displays the history list from event 1030 through event 1035:

```
$ fc -l 1030 1035
```

```
1030 cd ../memos
```

```
1031 ls
```

```
1032 rm *0405
```

```
1033 fc -l
```

```
1034 cd
```

```
1035 whereisaspell
```

This command lists the most recent event that begins with view through the most recent command line that begins with whereis: **\$ fc -l view whereis**

```
1025 view calendar
```

```
1026 vim letter.adams01
```

```
1027 aspell -c letter.adams01
```

```
1028 vim letter.adams01
```

```
1029 lpr letter.adams01
```

```
1030 cd ../memos
```

```
1031 ls
```

```
1032 rm *0405
```

```
1033 fc -l
```

```
1034 cd
```

```
1035 whereisaspell
```

To list a single command from the history list, use the same identifier for the first and second arguments. The following command lists event 1027: **\$ fc -l 1027 1027**

```
1027 ell -c letter.adams01
```

You can use **fc** to edit and reexecute previous commands: **fc [-e editor] [first [last]]**. When you call **fc** with the **-e** option followed by the name of an editor, **fc** calls the editor with event(s) in the Work buffer. By default, **fc** invokes the nano editor. Without first and last, it defaults to the most recent command: **\$ fc -e vi**. The **fc** builtin uses the stand-alone vim editor. If you set the **FCEDIT** variable, you do not need to use the **-e** option to specify an editor on the command line. Because the value of **FCEDIT** has been changed to **/usr/bin/emacs** and **fc** has no arguments, the following command edits the most recent command using the emacs editor:

```
$ export FCEDIT=/usr/bin/emacs
```

```
$ fc
```

If you call it with a single argument, **fc** invokes the editor on the specified command. This command starts the editor with event 1029 in the Work buffer. When you exit from the editor, the shell executes the command: **\$ fc 1029**. You can identify commands with numbers or by specifying the first few characters of the command name. This command calls the editor to work on events from the most recent event that begins with the letters vim through event 1030: **\$ fc vim 1030**. You can reexecute previous commands without using an editor. If you call **fc** with the **-s** option, it skips the editing phase and reexecutes the command. This command reexecutes event 1029: **\$ fc -s 1029**

```
lpr letter.adams01
```

This command reexecutes the previous command:

```
$ fc -s
```

When you reexecute a command, you can tell `fc` to substitute one string for another. This command substitutes the string `john` for the string `adams` in event 1029 and executes the modified event:

```
$ fc -s adams=john 1029
```

```
lpr letter.john01
```

Using an Exclamation Point (!) to Reference Events

The C Shell history mechanism uses an exclamation point to reference events. For example, the **!!** command reexecutes the previous event, and the shell replaces the `!$` token with the last word on the previous command line. You can reference an event by using its absolute event number, its relative event number, or the text it contains. All references to events, called event designators, begin with an exclamation point (!). One or more characters follow the exclamation point to specify an event. You can put history events anywhere on a command line. To escape an exclamation point so that the shell interprets it literally instead of as the start of a history event, precede the exclamation point with a backslash (\) or enclose it within single quotation marks.

Event Designators

Designator	Meaning
!	Starts a history event unless followed immediately by SPACE, NEWLINE, = or (.
!!	The previous command
!n	Command number n in the history list
!-n	The nth preceding command
!string	The most recent command line that started with string
!?string[?]	The most recent command that contained string. The last ? is optional
!#	The current command
!{event}	The event is an event designator. The braces isolate event from the surrounding text.

An event designator specifies a command in the history list.

You can reexecute the previous event by giving a **!!** command. In the following example, event 45 reexecutes event 44:

```
44 $ ls -l text
```

```
-rw-rw-r-- 1 max group 45 Apr 30 14:53 text
```

```
45 $ !!
ls -l text
-rw-rw-r-- 1 max group 45 Apr 30 14:53 text
```

A number following an exclamation point refers to an event. If that event is in the history list, the shell executes it. Otherwise, the shell displays an error message. A negative number following an exclamation point references an event relative to the current event. For example, the command `!-3` refers to the third preceding event. After you issue a command, the relative event number of a given event changes (event `-3` becomes event `-4`). Both of the following commands reexecute event 44:

```
51 $ !44
ls -l text
-rw-rw-r-- 1 max group 45 Apr 30 14:53 text

52 $ !-8
ls -l text
-rw-rw-r-- 1 max group 45 Apr 30 14:53 text
```

When a string of text follows an exclamation point, the shell searches for and executes the most recent event that began with that string. If you enclose the string within question marks, the shell executes the most recent event that contained that string. The final question mark is optional if a RETURN would immediately follow it.

The Readline Library

Command-line editing under the Bourne Again Shell is implemented through the Readline Library, which is available to any application written in C. Any application that uses the Readline Library supports line editing that is consistent with that provided by bash. Programs that use the Readline Library, including bash, read `~/inputrc` for key binding information and configuration settings. The `--noediting` command-line option turns off command-line editing in bash. You can choose one of two editing modes when using the Readline Library in bash: emacs or vi(m). Both modes provide many of the commands available in the standalone versions of the emacs and vim editors. You can also use the ARROW keys to move around. Up and down movements move you backward and forward through the history list. In addition, Readline provides several types of interactive word completion. The default mode is emacs; you can switch to vi mode with the following command: `$ set -o vi`. This command switches back to emacs mode: `$ set -o emacs`

Before you start, make sure the shell is in vi mode. When you enter bash commands while in vi editing mode, you are in Input mode. As you enter a command, if you discover an error before you press RETURN, you can press ESCAPE to switch to vim Command mode.

Unlike the vim editor, emacs is modeless. You need not switch between Command mode and Input mode because most emacs commands are control characters, allowing emacs to distinguish between input and commands. Like vim, the emacs command-line editor provides commands for moving the cursor on the command line and through the command history list and for modifying part or all of a command. However, in a few cases, the emacs command-line editor commands differ from those in the stand-alone emacs editor. In emacs you perform cursor movement by using both CONTROL and ESCAPE commands.

You can use the TAB key to complete words you are entering on the command line. This facility, called completion, works in both vi and emacs editing modes and is similar to the completion facility available in tcsh. Several types of completion are possible, and which one you use depends on which part of a command line you are typing when you press TAB.

Command Completion

If you are typing the name of a command (usually the first word on the command line), pressing TAB initiates command completion, in which bash looks for a command whose name starts with the part of the word you have typed. If no command starts with the characters you entered, bash beeps. If there is one such command, bash completes the command name. If there is more than one choice, bash does nothing in vi mode and beeps in emacs mode. Pressing TAB a second time causes bash to display a list of commands whose names start with the prefix you typed and allows you to continue typing the command name. In the following example, the user types bz and presses TAB. The shell beeps (the user is in emacs mode) to indicate that several commands start with the letters bz. The user enters another TAB to cause the shell to display a list of commands that start with bz followed by the command line as the user had entered it so far:

```
$ bz → TAB (beep) → TAB
```

```
bzcatbzdif bzip2 bzless
```

```
bzcmpbzgrep bzip2recover bzmore
```

```
$ bz■
```

Next the user types c and presses TAB twice. The shell displays the two commands that start with bzc. The user types a followed by TAB. At this point the shell completes the command because only one command starts with bzca.

```
$ bzc → TAB (beep) → TAB
```

```
bzcatbzcmp
```

```
$ bzca → TAB →t ■
```

Pathname Completion

Pathname completion, which also uses TABs, allows you to type a portion of a pathname and have bash supply the rest. If the portion of the pathname you have typed is sufficient to determine a unique pathname, bash displays that pathname. If more than one pathname would match it, bash completes the pathname up to the point where there are choices so that you can type more. When you are entering a pathname, including a simple filename, and press TAB, the shell beeps (if the shell is in emacs mode—in vi mode there is no beep). It then extends the command line as far as it can.

```
$ cat films/dar → TAB (beep) cat films/dark_
```

In the films directory every file that starts with dar has k_ as the next characters, so bash cannot extend the line further without making a choice among files. The shell leaves the cursor just past the _ character. At this point you can continue typing the pathname or press TAB twice. In the latter case bash beeps, displays your choices, redisplay the command line, and again leaves the cursor just after the _ character.

```
$ cat films/dark_ → TAB (beep) → TAB
```

```
dark_passagedark_victory
```

```
$ cat films/dark_
```

When you add enough information to distinguish between the two possible files and press TAB, bash displays the unique pathname. If you enter p followed by TAB after the _ character, the shell completes the command line: \$ cat films/dark_p → TAB → assage. Because there is no further ambiguity, the shell appends a SPACE so you can finish typing the command line or just press RETURN to execute the command. If the complete pathname is that of a directory, bash appends a slash (/) in place of a SPACE.

Variable Completion

When you are typing a variable name, pressing TAB results in variable completion, wherein bash attempts to complete the name of the variable. In case of an ambiguity, pressing TAB twice displays a list of choices:

```
$ echo $HO → TAB → TAB
```

```
$HOME $HOSTNAME $HOSTTYPE
```

```
$ echo $HOM → TAB → E
```

.inputrc: Configuring the Readline Library

The Bourne Again Shell and other programs that use the Readline Library read the file specified by the INPUTRC environment variable to obtain initialization information. If INPUTRC is not set, these programs read the ~/.inputrc file. They ignore lines of .inputrc that are blank or that start with a pound sign (#). You can set variables in .inputrc to control the behavior of the Readline Library using the following syntax: **set variable value**.

Readline variables

Variable	Effect
editing-mode	Set to vi to start Readline in vi mode. Set to emacs to start Readline in emacs mode (the default). Similar to the set -o vi and set -o emacs shell commands
horizontal-scroll-mode	Set to on to cause long lines to extend off the right edge of the display area. Moving the cursor to the right when it is at the right edge of the display area shifts the line to the left so you can see more of the line. You can shift the line back by moving the cursor back past the left edge. The default value is off, which causes long lines to wrap onto multiple lines of the display.
mark-directories	Set to off to cause Readline not to place a slash (/) at the end of directory names it completes. The default value is on.
mark-modified-lines	Set to on to cause Readline to precede modified history lines with an asterisk. The default value is off.

8.11 Alias

An alias is a usually short name that the shell translates into another usually longer name or complex command. Aliases allow you to define new commands by substituting a string for the first token of a simple command. They are typically placed in the ~/.bashrc (bash) or ~/.tcshrc (tcsh) startup files so that they are available to interactive subshells. Under bash the syntax of the alias builtin is: **alias [name[=value]]**. Under tcsh the syntax is: **alias [name[value]]**.

In the bash syntax no SPACES are permitted around the equal sign. If value contains SPACES or TABs, you must enclose value within quotation marks. Unlike aliases under tcsh, a bash alias does not accept an argument from the command line in value. Use a bash function when you need to use an argument. An alias does not replace itself, which avoids the possibility of infinite recursion in handling an alias such as the following: **\$ alias ls='ls -F'**. You can nest aliases. Aliases are disabled for noninteractive shells (that is, shell scripts). To see a list of the current aliases, give the command **alias**. To view the alias for a particular name, give the command **alias** followed by the name of the alias. You can use the **unalias** builtin to remove an alias. When you give an alias builtin command without any arguments, the shell displays a list of all defined aliases:

```
$ alias
alias ll='ls -l'
alias l='ls -ltr'
alias ls='ls -F'
alias zap='rm -i'
```

Give an alias command to see which aliases are in effect. You can delete the aliases you do not want from the appropriate startup file.

Single Versus Double Quotation Marks in Aliases

The choice of single or double quotation marks is significant in the alias syntax when the alias includes variables. If you enclose value within double quotation marks, any variables that appear in value are expanded when the alias is created. If you enclose value within single quotation marks, variables are not expanded until the alias is used. The PWD keyword variable holds the pathname of the working directory (Shown in next slide). Max creates two aliases while he is working in his home directory. Because he uses double quotation marks when he creates the dirA alias, the shell substitutes the value of the working directory when he creates this alias. The alias dirA command displays the dirA alias and shows that the substitution has already taken place:

```
$ echo $PWD
/home/max
$ alias dirA="echo Working directory is $PWD"
$ alias dirA
alias dirA='echo Working directory is /home/max'
```

When Max creates the dirB alias, he uses single quotation marks, which prevent the shell from expanding the \$PWD variable. The alias dirB command shows that the dirB alias still holds the unexpanded \$PWD variable:

```
$ alias dirB='echo Working directory is $PWD'
$ alias dirB
alias dirB='echo Working directory is $PWD'
```

After creating the dirA and dirB aliases, Max uses cd to make cars his working directory and gives each of the aliases as commands. The alias he created using double quotation marks displays the name of the directory he created the alias in as the working directory (which is wrong). In contrast, the dirB alias displays the proper name of the working directory.

```
$ cd cars
$ dirA
Working directory is /home/max
$ dirB
Working directory is /home/max/cars
```

Examples of Aliases

The following alias allows you to type r to repeat the previous command: \$ alias r='fc -s'. If you use the command ls -ltr frequently, you can create an alias that substitutes ls -ltr when you give the command l:

```
$ alias l='ls -ltr'
```

```
$ l
total 41
-rw-r--r-- 1 max group 30015 Mar 1 2008 flute.ps
-rw-r----- 1 max group 3089 Feb 11 2009 XTerm.ad
-rw-r--r-- 1 max group 641 Apr 1 2009 fixtax.icn
-rw-r--r-- 1 max group 484 Apr 9 2009 maptax.icn
drwxrwxr-x 2 max group 1024 Aug 9 17:41 Tiger
drwxrwxr-x 2 max group 1024 Sep 10 11:32 testdir
-rwxr-xr-x 1 max group 485 Oct 21 08:03 floor
drwxrwxr-x 2 max group 1024 Oct 27 20:19 Test_Emacs
```

Another common use of aliases is to protect yourself from mistakes. The following example substitutes the interactive version of the rm utility when you give the command zap:

```
$ alias zap='rm -i'
$ zap f*
rm: remove 'fixtax.icn'? n
rm: remove 'flute.ps'? n
rm: remove 'floor'? n
```

The -i option causes rm to ask you to verify each file that would be deleted, thereby helping you avoid deleting the wrong file. You can also alias rm with the rm -i command: alias rm='rm -i'. This aliases cause the shell to substitute ls -l each time you give an ll command and ls -F each time you use ls:

```
$ alias ls='ls -F'
$ alias ll='ls -l'
```

```
$ ll
total 41
drwxrwxr-x 2 max group 1024 Oct 27 20:19 Test_Emacs/
drwxrwxr-x 2 max group 1024 Aug 9 17:41 Tiger/
-rw-r----- 1 max group 3089 Feb 11 2009 XTerm.ad
-rw-r--r-- 1 max group 641 Apr 1 2009 fixtax.icn
-rw-r--r-- 1 max group 30015 Mar 1 2008 flute.ps
-rwxr-xr-x 1 max group 485 Oct 21 08:03 floor*
-rw-r--r-- 1 max group 484 Apr 9 2009 maptax.icn
drwxrwxr-x 2 max group 1024 Sep 10 11:32 testdir/
```

The -F option causes ls to print a slash (/) at the end of directory names and an asterisk (*) at the end of the names of executable files. In this example, the string that replaces the alias ll (ls -l) itself contains an alias (ls). When it replaces an alias with its value, the shell looks at the first word of the replacement string to see whether it is an alias. In the preceding example, the replacement string contains the alias ls, so a second substitution occurs to produce the final command ls -F -l. When given a list of aliases without the =value or value field, the alias builtin responds by displaying the value of each defined alias. The alias builtin reports an error if an alias has not been defined:

The Linux and Shell Scripting

```
$ alias ll l lszapwx
alias ll='ls -l'
alias l='ls -ltr'
alias ls='ls -F'
alias zap='rm -i'
bash: alias: wx: not found
```

You can avoid alias substitution by preceding the aliased command with a backslash (\):

```
$ \ls
Test_Emacs XTerm.ad flute.ps maptax.icn
Tiger fixtax.icn floor testdir
```

Because the replacement of an alias name with the alias value does not change the rest of the command line, any arguments are still received by the command that gets executed:

```
$ ll f*
-rw-r--r-- 1 max group 641 Apr 1 2009 fixtax.icn
-rw-r--r-- 1 max group 30015 Mar 1 2008 flute.ps
-rwxr-xr-x 1 max group 485 Oct 21 08:03 floor*
```

You can remove an alias with the unalias builtin. When the zap alias is removed, it is no longer displayed with the alias builtin and its subsequent use results in an error message:

```
$ unalias zap
$ alias
alias ll='ls -l'
alias l='ls -ltr'
alias ls='ls -F'
$ zap maptax.icn
```

8.12 Functions

A bash shell function (tcsh does not have functions) is similar to a shell script in that it stores a series of commands for execution at a later time. However, because the shell stores a function in the computer's main memory (RAM) instead of in a file on the disk, the shell can access it more quickly than the shell can access a script. The shell also preprocesses (parses) a function so that it starts up more quickly than a script. Finally the shell executes a shell function in the same shell that called it. If you define too many functions, the overhead of starting a subshell can become unacceptable. You can declare a shell function in the ~/.bash_profile startup file, in the script that uses it, or directly from the command line. You can remove functions with the unset builtin. The shell does not retain functions after you log out.

Removing variables and functions

If you have a shell variable and a function with the same name, using unset removes the shell variable. If you then use unset again with the same name, it removes the function. The syntax that declares a shell function is

```
[function] function-name ()
{
```

commands

}

The word `function` is optional, `function-name` is the name you use to call the function, and `commands` comprise the list of commands the function executes when you call it. The commands can be anything you would include in a shell script, including calls to other functions. There are some features of functions:

- The opening brace (`{`) can appear on the same line as the function name.
- Aliases and variables are expanded when a function is read, not when it is executed.
- You can use the `break` statement within a function to terminate its execution.
- Shell functions are useful as a shorthand as well as to define special commands.
- It creates a simple function that displays the date, a header, and a list of the people who are logged in on the system. This function runs the same commands as the `whoson` script.
- The greater than (`>`) signs are secondary shell prompts (`PS2`); do not enter them.

```
$ function whoson ()
```

```
> {
```

```
> date
```

```
> echo "Users Currently Logged On"
```

```
> who
```

```
> }
```

```
$ whoson
```

```
Sun Aug 9 15:44:58 PDT 2009
```

```
Users Currently Logged On
```

```
hls console Aug 8 08:59 (:0)
```

```
max pts/4 Aug 8 09:33 (0.0)
```

```
zach pts/7 Aug 8 09:23 (bravo.example.com)
```

Functions in startup files

If you want to have the `whoson` function always be available without having to enter it each time you log in, put its definition in `~/ .bash_profile`. Then run `.bash_profile`, using the `.` (`dot`) command to put the changes into effect immediately:

```
$ cat ~/.bash_profile
```

```
export TERM=vt100
```

```
stty kill '^u'
```

```
whoson ()
```

```
{
```

```
date
```

```
echo "Users Currently Logged On"
```

```
who
```

```
}
```

```
$ . ~/.bash_profile
```

The Linux and Shell Scripting

You can specify arguments when you call a function. Within the function these arguments are available as positional parameters. The following example shows the `arg1` function entered from the keyboard:

```
$ arg1 () {
> echo "$1"
> }
$ arg1 first_arg
first_arg
```

8.13 Command Line Options

Two kinds of command-line options are available: short and long. Short options consist of a hyphen followed by a letter. Long options have two hyphens followed by multiple characters. Long options must appear before short options on a command line that calls `bash`.

Option	Explanation	Syntax
Help	Displays a usage message.	<code>--help</code>
No edit	Prevents users from using the Readline Library to edit command lines in an interactive shell.	<code>--noediting</code>
No profile	Prevents reading these startup files: <code>/etc/profile</code> , <code>~/.bash_profile</code> , <code>~/.bash_login</code> , and <code>~/.profile</code> .	<code>--noprofile</code>
No rc	Prevents reading the <code>~/.bashrc</code> startup file. This option is on by default if the shell is called as sh .	<code>--norc</code>
POSIX	Runs <code>bash</code> in POSIX mode.	<code>--posix</code>
Version	Displays <code>bash</code> version information and exits.	<code>--version</code>
Login	Causes <code>bash</code> to run as though it were a login shell.	<code>-l</code> (lowercase "l")
Shopt	Runs a shell with the <code>opt shopt</code> option. A <code>-O</code> (uppercase "O") sets the option; <code>+O</code> unsets it.	<code>[±] 0 [opt]</code>
End of options	On the command line, signals the end of options. Subsequent tokens are treated as arguments even if they begin with a hyphen (<code>-</code>).	<code>--</code>

8.14 Shell Features

You can control the behavior of the Bourne Again Shell by turning features on and off. Different features use different methods to turn features on and off. The `set` builtin controls one group of features, while the `shopt` builtin controls another group. You can also control many features from the command line you use to call `bash`.

set ±o: Turns Shell Features On and Off

The `bash set` builtin (there is a `set` builtin in `tcsh`, but it works differently), when used with the `-o` or `+o` option, enables, disables, and lists certain `bash` features. For example, the following command turns on the `noclobber` feature: `$ set -o noclobber`. You can turn this feature off (the default) by giving the command: `$ set +o noclobber`. The command `set -o` without an option lists each of the features controlled by `set`, followed by its state (on or off). The command `set +o` without an option lists the same features in a form you can use as input to the shell.

shopt: Turns Shell Features On and Off

The `shopt` (shell option) builtin (not available in `tcsh`) enables, disables, and lists certain `bash` features that control the behavior of the shell. For example, the following command causes `bash` to include filenames that begin with a period (.) when it expands ambiguous file references (the `-s` stands for `set`): `$ shopt -s dotglob`. You can turn this feature off (the default) by giving the following command (the `-u` stands for `unset`): `$ shopt -u dotglob`. The shell displays how a feature is set if you give the name of the feature as the only argument to `shopt`: `$ shopt dotglob` displays `dotglob off`. The command `shopt` without any options or arguments lists the features controlled by `shopt` and their state. The command `shopt -s` without an argument lists the features controlled by `shopt` that are set or on. The command `shopt -u` lists the features that are unset or off.

bash features

Feature	Description	Syntax	Alternate Syntax
<code>allexport</code>	Automatically exports all variables and functions you create or modify after giving this command.	<code>set -o allexport</code>	<code>set -a</code>
<code>braceexpand</code>	Causes <code>bash</code> to perform brace expansion	<code>set -o braceexpand</code>	<code>set -B</code>
<code>cdspell</code>	Corrects minor spelling errors in directory names used as arguments to <code>cd</code> .	<code>shopt -s cdspell</code>	
<code>cmdhist</code>	Saves all lines of a multiline command in the same history entry, adding semicolons as needed.	<code>shopt -s cmdhist</code>	

The Linux and Shell Scripting

dotglob	Causes shell special characters (wildcards) in an ambiguous file reference to match a leading period in a filename. By default, special characters do not match a leading period. You must always specify the filenames <code>.</code> and <code>..</code> explicitly because no pattern ever matches them.	shopt -s dotglob	
emacs	Specifies emacs editing mode for command-line editing	set -o emacs	
errexit	Causes bash to exit when a simple command (not a control structure) fails.	set -o errexit	set -e
execfail	Causes a shell script to continue running when it cannot find the file that is given as an argument to <code>exec</code> . By default a script terminates when <code>exec</code> cannot find the file that is given as its argument.	shopt -s execfail	
expand_aliases	Causes aliases to be expanded (by default it is on for interactive shells and off for noninteractive shells).	shopt -s expand_alias	
hashall	Causes bash to remember where commands it has found using PATH are located (default).	set -o hashall	set -h
histappend	Causes bash to append the history list to the file named by HISTFILE when the shell exits. By default bash overwrites this file.	shopt -s histappend	
histexpand	Turns on the history mechanism (which uses exclamation points by default. Turn this feature off to turn off history expansion.	set -o histexpand	set -H
history	Enables command history	set -o history	
huponexit	Specifies that bash send a SIGHUP signal to all jobs when an interactive login shell exits.	shopt -s huponexit	
ignoreeof	Specifies that bash must receive ten EOF characters before it exits. Useful on noisy dial-up lines.	set -o ignoreeof	

Unit 08: The Bourne Again Shell and TC Shell

monitor	Enables job control	set -o monitor	set -m
nocaseglob	Causes ambiguous file references to match filenames without regard to case (off by default).	shopt -s nocaseglob	
noclobber	Helps prevent overwriting files	set -o noclobber	set -C
noglob	Disables pathname expansion	set -o noglob	set -f
notify	With job control enabled, reports the termination status of background jobs immediately. The default behavior is to display the status just before the next prompt.	set -o notify	set -b
nounset	Displays an error and exits from a shell script when you use an unset variable in an interactive shell. The default is to display a null value for an unset variable.	set -o nounset	set -u
nullglob	Causes bash to expand ambiguous file references that do not match a filename to a null string. By default bash passes these file references without expanding them.	shopt -s nullglob	
posix	Runs bash in POSIX mode.	set -o posix	
verbose	Displays command lines as bash reads them.	set -o verbose	set -v
vi	Specifies vi editing mode for commandline editing	set -o vi	
xpg_echo	Causes the echo builtin to expand backslash escape sequences without the need for the -e option	shopt -s xpg_echo	
xtrace	Turns on shell debugging	set -o xtrace	set -x

8.15 The TC Shell

The TC Shell (tcsh) provides an interface between you and the Linux operating system. The TC Shell is an interactive command interpreter as well as a high-level programming language. You use only one shell at any given time. The TC Shell is an expanded version of the C Shell. The “T” in TC Shell comes from the TENEX and TOPS-20 operating systems. A number of features not found in

The Linux and Shell Scripting

csd are present in tcsh, including file and username completion, command-line editing, and spelling correction.

Assignment statement

The tcsh assignment statement has the following syntax: `set variable = value`. Having SPACES on either side of the equal sign, although illegal in bash, is allowed in tcsh. The default tcsh prompt is a greater than sign (`>`), but it is frequently set to a single `$` character followed by a SPACE.

Shell Scripts

The TC Shell can execute files containing tcsh commands. If the first character of a shell script is a pound sign (`#`) and the following character is not an exclamation point (`!`), the TC Shell executes the script under tcsh. If the first character is anything other than `#`, tcsh calls the sh link to dash or bash to execute the script. The tcsh echo builtin accepts either a `-n` option or a trailing `\c` to get rid of the RETURN that echo normally displays at the end of a line.

Checking shell

- **ps:** If you are not sure which shell you are using, use the ps utility to find out. It shows whether you are running tcsh, bash, sh (linked to bash), or possibly another shell.
- **finger:** The finger command followed by your username displays the name of your login shell, which is stored in the `/etc/passwd` file.

Entering the TC Shell

You can execute tcsh by giving the command `tcsh`. If you want to use tcsh as a matter of course, you can use the `chsh` (change shell) utility to change your login shell:

```
bash $ chsh
```

Changing shell for sam.

Password:

```
New shell [/bin/bash]: /bin/tcsh
```

Shell changed.

```
bash $
```

Leaving the tc shell

You can leave tcsh in several ways. The approach you choose depends on two factors: whether the shell variable `ignoreeof` is set and whether you are using the shell that you logged in on (your login shell) or another shell that you created after you logged in. If you are not sure how to exit from tcsh, press `CONTROL-D` on a line by itself with no leading SPACES, just as you would to terminate standard input to a program. You will either exit or receive instructions on how to exit. If you have not set `ignoreeof` and it has not been set for you in a startup file, you can exit from any shell by using `CONTROL-D` (the same procedure you use to exit from the Bourne Again Shell). When `ignoreeof` is set, `CONTROL-D` does not work. The `ignoreeof` variable causes the shell to display a message telling you how to exit. You can always exit from tcsh by giving an exit command. A `logout` command allows you to exit from your login shell only.

Startup Files

When you log in on the TC Shell, it automatically executes various startup files. You must have read access to a startup file to execute the commands in it. When you log in on the TC Shell, it automatically executes various startup files. You must have read access to a startup file to execute the commands in it.

1) `/etc/csh.cshrc` and `/etc/csh.login`:

The shell first executes the commands in `/etc/csh.cshrc` and `/etc/csh.login`. A user working with root privileges can set up these files to establish systemwide default characteristics for tcsh users. They contain systemwide configuration information, such as the default path, the location to check for mail, and so on.

2) `.tcshrc` and `.cshrc`:

Next the shell looks for `~/ .tcshrc` or, if it does not exist, `~/ .cshrc`. You can use these files to establish variables and parameters that are local to your shell. Each time you create a new shell, tcsh reinitializes these variables for the new shell.

3) `.history`

Login shells rebuild the history list from the contents of `~/ .history`. If the `histfile` variable exists, tcsh uses the file that `histfile` points to in place of `.history`.

4) `.login`

Login shells read and execute the commands in `~/ .login`. This file contains commands that you want to execute once, at the beginning of each session.

5) `/etc/csh.logout` and `.logout`

The TC Shell runs the `/etc/csh.logout` and `~/ .logout` files, in that order, when you exit from a login shell.

Features Common to the Bourne Again and TC Shells

Most of the features common to both bash and tcsh are derived from the original C Shell:

- Command-line expansion (also called substitution)
- History
- Aliases
- Job control
- Filename substitution
- Directory stack manipulation
- Command substitution

Command-Line Expansion (Substitution)

The tcsh man page uses the term substitution instead of expansion; the latter is used by bash. The TC Shell scans each token for possible expansion in the following order:

The Linux and Shell Scripting

1. History substitution
2. Alias substitution
3. Variable substitution
4. Command substitution
5. Filename substitution
6. Directory stack substitution

1) History substitution:

The TC Shell assigns a sequential event number to each command line. You can display this event number as part of the tcsh prompt. As in bash, the tcsh history builtin displays the events in your history list. The list of events is ordered with the oldest events at the top. The last event in the history list is the history command that displayed the list. In the following history list, which is limited to ten lines by the argument of 10 to the history command, command 23 modifies the tcsh prompt to display the history event number. The time each command was executed appears to the right of the event number.

```
32 $ history 10
```

```
23 23:59 set prompt = "! $ "
```

```
24 23:59 ls -l
```

```
25 23:59 cat temp
```

```
26 0:00 rm temp
```

```
27 0:00 vim memo
```

```
28 0:00 lpr memo
```

```
29 0:00 vim memo
```

```
30 0:00 lpr memo
```

```
31 0:00 rm memo
```

```
32 0:00 history
```

The same event and word designators work in both shells. For example, !! refers to the previous event in tcsh, just as it does in bash. The command !328 executes event number 328; !?txt? executes the most recent event containing the string txt.

- Few tcsh word modifiers not found in bash.

Modifier	Function
u	Converts the first lowercase letter into uppercase
l	Converts the first uppercase letter into lowercase
a	Applies the next modifier globally within a single word

- Variables to control history

Variable	Default	Function
history	100 words	Maximum number of events saved during a session
histfile	~/ .history	Location of the history file
savehist	not set	Maximum number of events saved between sessions

2) Aliases Substitution

The alias builtin has a slightly different syntax: alias name value. The following command creates an alias for ls: tcsh \$ alias ls "ls -lF". The tcsh alias allows you to substitute command-line arguments, whereas bash does not:

```
$ alias nam "echo Hello, \!^ is my name"
```

```
$ nam Sam
```

```
Hello, Sam is my name
```

The string \!* within an alias expands to all command-line arguments: \$ alias sortprint "sort \!* | lpr"

The next alias displays its second argument: \$ alias n2 "echo \!:2"

3) Job Control

Job control is similar in both bash and tcsh. You can move commands between the foreground and the background, suspend jobs temporarily, and get a list of the current jobs. The % character references a job when it is followed by a job number or a string prefix that uniquely identifies the job. You will see a minor difference when you run a multiple-process command line in the background from each shell. Whereas bash displays only the PID number of the last background process in each job, tcsh displays the numbers for all processes belonging to a job.

4) Filename Substitution

The TC Shell expands the characters *, ?, and [] in a pathname just as bash does. * matches any string of zero or more characters, ? matches any single character, [] defines a character class, which is used to match single characters appearing within a pair of brackets. The TC Shell expands command-line arguments that start with a tilde (~) into filenames in much the same way that bash does, with the ~ standing for the user's home directory or the home directory of the user whose name follows the tilde. The bash special expansions ~+ and ~- are not available in tcsh. Brace expansion is available in tcsh. Like tilde expansion, it is regarded as an aspect of filename substitution even though brace expansion can generate strings that are not the names of actual files.

5) Manipulating the Directory Stack

Directory stack manipulation in tcsh does not differ much from that in bash. The dirs builtin displays the contents of the stack, and the pushd and popdbuiltins push directories onto and pop directories off of the stack.

6) Command Substitution

The `$(...)` format for command substitution is not available in `tcsh`. In its place you must use the original `'...'` format. Otherwise, the implementation in `bash` and `tcsh` is identical.

Summary

- The Bourne Again Shell and TC Shell are command interpreters and high-level programming languages.
- Login shells are, by their nature, interactive.
- Pressing the suspend key (usually CONTROL-Z) immediately suspends (temporarily stops) the job in the foreground and displays a message that includes the word Stopped.
- Keyword shell variables have special meaning to the shell and usually have short, mnemonic names.
- Like the file structure, the process structure is hierarchical, with parents, children, and even a root. A parent process forks a child process, which in turn can fork other processes.
- You can declare a shell function in the `~/.bash_profile` startup file, in the script that uses it, or directly from the command line.
- You can remove functions with the `unset` builtin. The shell does not retain functions after you log out.
- The `shopt` (shell option) builtin (not available in `tcsh`) enables, disables, and lists certain `bash` features that control the behavior of the shell.
- The TC Shell is an interactive command interpreter as well as a high-level programming language.

Keywords

- **BASH_ENV:** Noninteractive shells look for the environment variable `BASH_ENV` (or `ENV` if the shell is called as `sh`) and execute commands in the file named by this variable.
- **File Descriptors:** A file descriptor is the place a program sends its output to and gets its input from.
- **Shell Script:** A shell script is a file that holds commands that the shell can execute. The commands in a shell script can be any commands you can enter in response to a shell prompt.
- **Positional Parameters:** Positional parameters enable you to access command-line arguments, a capability that you will often require when you write shell scripts.
- **MAIL:** The `MAIL` variable (mail under `tcsh`) contains the pathname of the file that holds your mail (your mailbox, usually `/var/mail/name`, where `name` is your username).
- **MAILPATH:** The `MAILPATH` variable (not available under `tcsh`) contains a list of filenames separated by colons. If this variable is set, the shell informs you when any one of the files is modified.
- **MAILCHECK:** This variable (not available under `tcsh`) specifies how often, in seconds, the shell checks for new mail. The default is 60 seconds. If you set this variable to zero, the shell checks before each prompt.
- **Process:** A process is the execution of a command by the Linux kernel.
- **Events:** The history mechanism, a feature adapted from the C Shell, maintains a list of recently issued command lines, also called events.

Unit 08: The Bourne Again Shell and TC Shell

- **HISTSIZE:** The value of the HISTSIZE variable determines the number of events preserved in the history list during a session. A value in the range of 100 to 1,000 is normal.
- **HISTFILE:** When you exit from the shell, the most recently executed commands are saved in the file whose name is stored in the HISTFILE variable (the default is ~/.bash_history). The next time you start the shell, this file initializes the history list.
- **HISTFILESIZE:** The value of the HISTFILESIZE variable determines the number of lines of history saved in HISTFILE.
- **Alias:** An alias is a usually short name that the shell translates into another usually longer name or complex command.
- **ps:** If you are not sure which shell you are using, use the ps utility to find out. It shows whether you are running tcsh, bash, sh (linked to bash), or possibly another shell.
- **finger:** The finger command followed by your username displays the name of your login shell, which is stored in the /etc/passwd file.

Self Assessment

1. The shoptbuiltin _____ the features that control the bash.
A. Enables
B. Disables
C. Lists
D. All of the above mentioned
2. The short command line options consists of
A. Hyphen
B. A letter
C. Hyphen followed by a letter
D. None of the above
3. Using _____, we can recall, modify and re-execute previously executed events.
A. Fc builtin
B. Exclamation point commands
C. Readline libraries
D. All of the above mentioned
4. Event designators start with
A. &
B. !
C. @
D. #
5. Which key is used for pathname and command completion?
A. CTRL
B. TAB
C. RETURN

- D. SHIFT
-
- 6. Which of these variables gives the location of history file?
 - A. HISTSIZE
 - B. HISTFILE
 - C. HISTFILESIZE
 - D. None of the above
-
- 7. Which builtin sets the attributes and values for shell variables?
 - A. declare
 - B. typeset
 - C. Both declare and typeset
 - D. None of the above mentioned
-
- 8. PS4 is _____
 - A. Primary prompt
 - B. Secondary prompt
 - C. Prompt issued by select
 - D. Bash debugging symbol
-
- 9. What is the naming convention for global variables?
 - A. Only lowercase letters
 - B. Only uppercase letters
 - C. Mixed case letters
 - D. Only numbers
-
- 10. What is the correct syntax for assigning a value to a variable in Bourne Again Shell?
 - A. VARIABLE=value
 - B. VARIABLE = value
 - C. VARIABLE= value
 - D. VARIABLE =value
-
- 11. Which keyword holds the pathname of the working directory?
 - A. pwd
 - B. work
 - C. dir
 - D. key
-
- 12. You can specify arguments when you call a function. Within the function these arguments are available as _____
 - A. Special parameters
 - B. Positional parameters
 - C. Uni parameters

- D. None of the above
13. Which built in is used to make the value of a variable available to the child processes?
- A. echo
 - B. export
 - C. cat
 - D. avail
14. What is used to see the parent-child relationship?
- A. pstree
 - B. treeps
 - C. trpsee
 - D. eespstr
15. The directory stack implements _____ rule.
- A. FIFO
 - B. LIFO
 - C. RIRO
 - D. None of the above
16. To remove a directory from the stack, use the _____ builtin.
- A. popd
 - B. pushd
 - C. remd
 - D. None of the above
17. Which variable gives the maximum number of events saved between the session?
- A. history
 - B. histfile
 - C. savehist
 - D. None of the above
18. Which of these features are common in bash and tcsh?
- A. Aliases
 - B. Job control
 - C. Command substitution
 - D. All of the above mentioned

Answers for Self Assessment

- | | | | | |
|-------|-------|-------|-------|-------|
| 1. D | 2. C | 3. D | 4. B | 5. B |
| 6. B | 7. C | 8. D | 9. B | 10. A |
| 11. A | 12. B | 13. B | 14. A | 15. A |
| 16. A | 17. C | 18. D | | |

Review Questions

- 1) What are startup files? Explain in detail.
- 2) What are file descriptors? Explain.
- 3) How can we manipulate a directory stack? Explain the operations.
- 4) What are shell variables? Explain its types in details.
- 5) What are keyword variables? Explain with examples.
- 6) What is a process in Linux? Explain it.
- 7) What is history feature in Linux? Write its features. Which variables that control history?
- 8) How can we re-execute and edit commands? Write the different ways to do this.
- 9) What is an alias? Write its syntax. What is the use of single and double quotation marks in alias? Explain with example.
- 10) What are shell features? Write bash features.
- 11) What is TC shell? How can we enter and leave the TC shell? Write its startup files.
- 12) Write the features common to Bourne again shell and TC shell.

**Further Readings**

Mark G. Sobell, A Practical Guide to Linux Commands, Editors, and Shell Programming, Second Edition, Prentice Hall, Pearson Education, Inc.

**Web Links**

https://www.bottomupcs.com/file_descriptors.xhtml

<https://www.ibm.com/docs/en/aix/7.2?topic=concepts-shell-features>